VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
Faculty of Electrical and Electronics Engineering

ASIC Verification

EE5213

Exercise 1 Report

Counter Design and Parameterized Stack Verification

Student: Vu Tien Giang - 2570188

Instructor: TS. Tran Hoang Linh

March 31, 2026

1 Exercise 1: Counter Design and Verification

1.1 Objective

Design a 5-bit synchronous counter based on the specifications provided and write a testbench to verify its correctness against a target timing trace.

1.2 Design Specifications

- The counter is clocked on the rising edge of `clk`.
- `rst` is active high and asynchronously resets the output to 0.
- `cnt_in` and `cnt_out` are 5-bit signals.
- If `load` is high, the counter is loaded from `cnt_in` (takes priority over `en`).
- Otherwise, if `en` is high, `cnt_out` is incremented.

1.3 Implementation (`counter.v`)

```
`timescale 1ns / 1ps

module counter (
    input clk,
    input rst,
    input load,
    input en,
    input [4:0] cnt_in,
    output reg [4:0] cnt_out
);
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            cnt_out <= 5'b00000;
        end else if (load) begin
            cnt_out <= cnt_in;
        end else if (en) begin
            cnt_out <= cnt_out + 1;
        end
    end
endmodule
```

1.4 Testbench (`counter_tb.v`)

To match the precise timestamps detailed in the exercise, the testbench drives inputs securely synchronized around the clock edges.

```
`timescale 1ns / 1ps

module counter_tb;
    reg clk;
    reg rst, load, en;
    reg [4:0] cnt_in;
    wire [4:0] cnt_out;
```

```
8
9       counter uut (.clk(clk), .rst(rst), .load(load), .en(en), .cnt_in(
        cnt_in), .cnt_out(cnt_out));
10
11      initial begin
12          clk = 0;
13          forever #5 clk = ~clk;
14      end
15
16      initial begin
17          rst = 1; load = 0; en = 0; cnt_in = 0;
18
19          #12; rst = 0; load = 1; en = 1; cnt_in = 5'b10101;
20          #8;  $display("At time %0t rst=%b load=%b en=%b cnt_in=%b
        cnt_out=%b",
21                $time, rst, load, en, cnt_in, cnt_out);
22
23          #2;  rst = 0; load = 1; en = 1; cnt_in = 5'b01010;
24          #8;  $display("At time %0t rst=%b load=%b en=%b cnt_in=%b
        cnt_out=%b",
25                $time, rst, load, en, cnt_in, cnt_out);
26
27          #2;  rst = 0; load = 1; en = 1; cnt_in = 5'b11111;
28          #8;  $display("At time %0t rst=%b load=%b en=%b cnt_in=%b
        cnt_out=%b",
29                $time, rst, load, en, cnt_in, cnt_out);
30
31          #2;  rst = 1; load = 1; en = 1; cnt_in = 5'b11111;
32          #8;  $display("At time %0t rst=%b load=%b en=%b cnt_in=%b
        cnt_out=%b",
33                $time, rst, load, en, cnt_in, cnt_out);
34
35          #2;  rst = 0; load = 1; en = 1; cnt_in = 5'b11111;
36          #8;  $display("At time %0t rst=%b load=%b en=%b cnt_in=%b
        cnt_out=%b",
37                $time, rst, load, en, cnt_in, cnt_out);
38
39          #2;  rst = 0; load = 0; en = 1; cnt_in = 5'b11111;
40          #8;  $display("At time %0t rst=%b load=%b en=%b cnt_in=%b
        cnt_out=%b",
41                $time, rst, load, en, cnt_in, cnt_out);
42
43          #10; $finish;
44      end
45  endmodule
```

1.5 Simulation Results

The simulation log matches the specified requirements at the detailed time points.

```
1  At time 20000 rst=0 load=1 en=1 cnt_in=10101 cnt_out=10101
2  At time 30000 rst=0 load=1 en=1 cnt_in=01010 cnt_out=01010
3  At time 40000 rst=0 load=1 en=1 cnt_in=11111 cnt_out=11111
4  At time 50000 rst=1 load=1 en=1 cnt_in=11111 cnt_out=00000
5  At time 60000 rst=0 load=1 en=1 cnt_in=11111 cnt_out=11111
6  At time 70000 rst=0 load=0 en=1 cnt_in=11111 cnt_out=00000
```

2 Exercise 2: Stack Design and Verification

2.1 Objective

Create a parameterized LIFO (Last-In-First-Out) Stack design, present testing scenarios, and write a testbench to verify edge cases.

2.2 Design Specifications

- Parameterized for `WIDTH` and `DEPTH`. `DEPTH` is dynamically managed via `DEPTH_LOG2` (power of 2 constraint).
- Asynchronous high-active reset (`rst`).
- `en_wr` (write) and `en_rd` (read) synchronous to the rising edge of `clk`.
- Data written later corresponds to earlier reads (LIFO constraint).
- Ignores en_wr attempts when `full` flag is set.
- Ignores en_rd attempts when `empty` flag is set.

2.3 Implementation (`stack.v`)

```
`timescale 1ns / 1ps

module stack #(
    parameter WIDTH = 8,
    parameter DEPTH_LOG2 = 3
) (
    input clk, rst, en_wr, en_rd,
    input [WIDTH-1:0] data_wr,
    output reg [WIDTH-1:0] data_rd,
    output reg full, empty
);
    localparam MAX_DEPTH = 1 << DEPTH_LOG2;
    reg [WIDTH-1:0] mem [0:MAX_DEPTH-1];
    reg [DEPTH_LOG2:0] sp;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            sp <= 0; full <= 0; empty <= 1; data_rd <= 0;
        end else begin
            if (en_wr && !full) begin
                mem[sp] <= data_wr;
                sp <= sp + 1;
                empty <= 0;
                if (sp + 1 == MAX_DEPTH) full <= 1;
            end else if (en_rd && !empty) begin
                data_rd <= mem[sp - 1];
                sp <= sp - 1;
                full <= 0;
                if (sp - 1 == 0) empty <= 1;
            end
        end
```

```
    end
endmodule
```

2.4 Testbench (stack_tb.v)

```
`timescale 1ns / 1ps

module stack_tb;
    parameter WIDTH = 8;
    parameter DEPTH_LOG2 = 2; // Testing depth 4

    reg clk, rst, en_wr, en_rd;
    reg [WIDTH-1:0] data_wr;
    wire [WIDTH-1:0] data_rd;
    wire full, empty;

    stack #(.WIDTH(WIDTH), .DEPTH_LOG2(DEPTH_LOG2)) uut (
        .clk(clk), .rst(rst), .en_wr(en_wr), .en_rd(en_rd),
        .data_wr(data_wr), .data_rd(data_rd), .full(full), .empty(empty
    )
    );

    initial forever #5 clk = ~clk;

    initial begin
        rst = 1; en_wr = 0; en_rd = 0; data_wr = 0;
        #15; rst = 0; #10;

        $display("Pushing elements...");
        push_data(8'h11); push_data(8'h22); push_data(8'h33); push_data
    (8'h44);

        $display("Trying to push when full...");
        push_data(8'h55);
        if (full) $display("Stack is full, correctly blocked push.");

        $display("Popping elements...");
        pop_data(); pop_data(); pop_data(); pop_data();

        $display("Trying to pop when empty...");
        pop_data();
        if (empty) $display("Stack is empty, correctly blocked pop.");

        #20 $finish;
    end

    task push_data(input [WIDTH-1:0] val);
    begin
        @(posedge clk); en_wr = 1; data_wr = val;
        @(posedge clk); en_wr = 0;
        $display("Pushed %h | full=%b empty=%b", val, full, empty);
    end
    endtask

    task pop_data();
    begin
```

```
        @(posedge clk); en_rd = 1;
        @(posedge clk); en_rd = 0;
        $display("Popped %h | full=%b empty=%b", data_rd, full, empty);
    end
    endtask
endmodule
```

2.5 Simulation Results

```
Pushing elements...
Pushed 11 | full=0 empty=0
Pushed 22 | full=0 empty=0
Pushed 33 | full=0 empty=0
Pushed 44 | full=1 empty=0
Trying to push when full...
Pushed 55 | full=1 empty=0
Stack is full, correctly blocked push.
Popping elements...
Popped 44 | full=0 empty=0
Popped 33 | full=0 empty=0
Popped 22 | full=0 empty=0
Popped 11 | full=0 empty=1
Trying to pop when empty...
Popped 11 | full=0 empty=1
Stack is empty, correctly blocked pop.
```

3 Exercise 3: FSM Reachability Analysis

3.1 Objective

Implement the State Reachability Analysis algorithm for an arbitrary Finite State Machine (FSM) using C++. The program computes the set of reachable states given a set of initial states and a transition relation.

3.2 Design Specifications

- **Input:** Set of initial states (I), and a transition relation (R) mapping current states to next possible states.
- **Output:** The complete set of reachable states (ARS - All Reached States).
- **Algorithm Strategy:**
  - Track **All Reached States** (`ARS`) and **New Next States** (`NNS` or Frontier Set).
  - Iteratively compute the next states (`NS`) from the current frontier `NNS`.
  - Update the frontier as newly discovered states (`NNS` = `NS` - `ARS`).
  - Add the new frontier to the all reached states (`ARS` = `ARS` $\cup$ `NNS`).
  - Terminate when the frontier is empty.

3.3 Implementation (fsm_reachability.cpp)

```cpp
#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <algorithm>

typedef int State;

std::set<State> Reachability(
    const std::map<State, std::set<State>>& R,
    const std::set<State>& I
) {
    std::set<State> ARS = I;  // All Reached States
    std::set<State> NNS = I;  // New Next States (Frontier Set)

    while (true) {
        std::set<State> NS;
        for (State s : NNS) {
            if (R.find(s) != R.end()) {
                for (State next_s : R.at(s)) {
                    NS.insert(next_s);
                }
            }
        }

        std::set<State> new_NNS;
        std::set_difference(
            NS.begin(), NS.end(),
            ARS.begin(), ARS.end(),
            std::inserter(new_NNS, new_NNS.begin())
        );
        NNS = new_NNS;

        if (NNS.empty()) {
            break;
        }
        ARS.insert(NNS.begin(), NNS.end());
    }
    return ARS;
}

int main() {
    std::map<State, std::set<State>> transition_relation = {
        {0, {1, 5}}, {1, {2}}, {5, {5, 2}},
        {2, {3}}, {3, {4}}, {4, {}}
    };
    std::set<State> initial_states = {0};
    std::set<State> reachable_states = Reachability(transition_relation
    , initial_states);

    std::cout << "Final Reachable States (ARS): { ";
    for (State s : reachable_states) std::cout << "S" << s << " ";
    std::cout << "}\n";
    return 0;
}
```

3.4 Execution Results

```
Starting Reachability Analysis...
Initial States: { S0 }

Final Reachable States (ARS): { S0 S1 S2 S3 S4 S5 }
```